

Simple Reasoning and Knowledge States in a LSTM-Based Agent

Ryan Mukai

June 27, 2020

1 Introduction

This article presents research originally completed in mid-2020, focusing on the development of a simple form of self-awareness in an LSTM-based agent. At that time, large language models had not yet demonstrated significant introspective capabilities, and most work on self-awareness in AI was limited to architectural or hardware-level monitoring. Our goal was to introduce an agent capable of a simple form of self-awareness in an LSTM-based agent. Our goal is to introduce an agent capable of displaying its knowledge state, detecting contradictions, and explicitly requesting assistance from other agents when it encounters gaps in knowledge. This extends prior work in neural-symbolic learning by embedding self-awareness mechanisms into an LSTM-based reasoning agent.

By 2020, self-awareness in neural networks was primarily studied at the architectural, circuit, and parameter-tuning levels [5]. Our work differs from prior research by introducing self-awareness at the symbolic knowledge level—allowing an agent to:

- Maintain an explicit knowledge state and identify when it lacks information.
- Detect contradictions within its knowledge base and issue warnings.
- Engage in cooperative problem-solving by querying other agents.

While LSTMs had been applied to logical reasoning and neuro-symbolic tasks before [2, 7, 6], none of these prior studies explicitly incorporated knowledge-state self-awareness, agent cooperation, or contradiction detection into an LSTM-based model. Our study represents an incremental but novel contribution toward bridging neural-symbolic reasoning and AI self-awareness.

Comparison to Prior Work Prior studies on self-awareness in AI [5] primarily focused on architectural and circuit-level introspection, where models adjusted their parameters dynamically. However, these approaches did not address symbolic knowledge-state reasoning, where an agent explicitly monitors its understanding of logical relationships.

Neural-symbolic learning has been extensively studied [7, 3, 8], and LSTMs have been applied to symbolic reasoning tasks [6]. However, these prior approaches:

- Did not incorporate explicit self-awareness mechanisms for monitoring knowledge states.
- Did not explore multi-agent cooperation via self-awareness-driven help requests.
- Did not include contradiction detection within an LSTM-based neuro-symbolic reasoning framework.

This study bridges these gaps by integrating knowledge-state self-awareness, multi-agent cooperation, and contradiction detection into a single LSTM-based system. This represents a key step toward embedding self-awareness principles into practical neural-symbolic reasoning agents.

2 The Problem Agents are Trained to Solve

Agents are trained to reason on sentences involving syllogisms and the logical not($\sim$), and ($\&$), or ($\mid$), exclusive-or ($\wedge$), implication ($\implies$), and bidirectional implication ($\iff$) operators. Throughout this article propositions, which can be either True or False, are denoted by the capital letters A through J inclusive. We train agents to handle simple problems of the following forms.

1. Given the status of a propositional variable A as true, false, or unknown, respond to a question about its value.
2. Given a simple sentence such as $A \implies B$ and given A is true, answer that B is true if asked about B.
3. Given a simple sentence such as $A \implies B$ and given B is false, answer that A is false if asked about A.
4. Given the value of A and no other information, if asked about B indicate B is unknown.
5. Given contradictory statements, warn that the knowledge base is contradictory.
6. In addition to sentences of the form $A \implies B$, we include sentences of the form $(A \mid B)$, $(A \wedge B)$, $(A \iff B)$, and $(A \& B)$.

3 The Agent

The core of the agent consists of an LSTM neural network created using Keras [4] with Tensorflow [1] as a backend.

The network is a sequence-to-sequence network of the type that has been used for language translation, and the source code is based directly upon the sequence-to-sequence tutorial found at https://keras.io/examples/lstm_seq2seq/ although, instead of translating from English to French, we are using the sequence-to-sequence model to answer logical questions.

The inputs are logic sentences represented with one-hot encoding. These may contain repeated sentences. We use an example to illustrate. Suppose the list of sentences below, one-hot encoded, is our input.

1. A
2. A ==> B
3. A ==> B
4. A

When presented with the above, the network is expected to be able to recall the sentences in the order they were presented, without any repetitions, as follows:

1. A
2. A ==> B

It is possible during the course of operations that some statements presented to the agent may contain one or more repetitions as shown above, and the agent is trained, upon receiving the query “HELP”, to respond by condensing the knowledge list into just two sentences as shown above. This serves two purposes:

1. Agents presented with lots of potentially repeating information must have a concept of what a logical sentence is, with the ability to condense repeat occurrences and return exactly the sentences that belong to the knowledge base. This implies some knowledge of a sentence as a unit of thought.
2. When an agent is asked for help, it should be able to supply its knowledge base in response to such a query from another agent.

If the word “HELP” appears, then the environment sets a help flag and send the question “HELP” to another agent. The other agent’s response, the dump of its knowledge base, is sent back to the first agent, and the first agent tries to answer the question again. If the word “HELP” does not appear, then the other agent is not queried for its knowledge since the first agent’s answer is sufficient.

3.1 Agent Cooperation: Example 1

Suppose we have two agents, Agent 1 and Agent 2. Agent 1 is given the following knowledge:

- $A \implies B$

Agent 2 is given the following knowledge:

- A

If we ask Agent 1, “What is B?”, the following will occur.

1. Agent 1 indicates B is unknown and requests help.
2. The simulation environment, in response to seeing Agent 1 request help, sends a help query to Agent 2.
3. Agent 2 dumps its knowledge that A is true.
4. This knowledge is added to the knowledge base of Agent 1.
5. Agent 1 runs again and is able to finally conclude B is true.

In this example, we note the following:

1. Agent 1 lacks adequate information to determine B.
2. Agent 1, aware of its lack of knowledge, makes a request for help.
3. Agent 2 responds to a help query by dumping its knowledge state.
4. This allows Agent 1 to find the answer.

From this example:

1. Agents know whether or not they know an answer and requesting help when they do not.
2. Agents also have knowledge states that they can dump if they are asked for help.
3. In terms of maintaining an internal knowledge state, an agent knows how to purge repeat sentences and understands, to some extent, the idea of a sentence as a unit of knowledge since it can repeat units of knowledge when asked.

3.2 A Second Example

Agent 1 is given the following knowledge:

- $A \implies B$

Agent 2 is given the following knowledge:

- $\neg B$

If we ask Agent 1, “What is A?”, the following will occur.

1. Agent 1 indicates a is unknown and requests help.
2. The simulation environment, in response to seeing Agent 1 request help, sends a help query to Agent 2.
3. Agent 2 dumps its knowledge $\neg B$.
4. This knowledge is added to the knowledge base of Agent 1.
5. Agent 1 runs again and is able to finally conclude A is false.

4 Key Results

The network was found to yield an error rate of slightly less than 1% on the sample data at https://github.com/CoderRyan800/redesigned-octo-goggles/blob/master/logic_data_extended_00.tsv which we have made publicly available to facilitate peer review. In this tab-separated file, the columns are defined as follows:

1. The first column, or column 0, contains a set of logical statements separated by the logical and (&) operator. This is the input data to the network consisting of logical statements and a question about a variable.
2. The second column, or column 1, contains the same set condensed, without any repetitions. It also contains the same question about a variable as column 0. This column was used while experimenting with training without repeated sentences during earlier stages of this research, but it was ignored in the final work and column 0 with possible repetition was used instead.
3. The third column, or column 2, is much like the second column, but without the question. This column contains the correct answer to the “HELP” query.
4. The fourth column, or column 3, contains the answer to the question in column 0, which could be: True, False, Contradictory, or Unknown HELP!

The network is trained to respond to the question with the answer from column 3 (fourth column). It is also trained to respond to the sentences, but with the question replaced by “HELP”, by yielding a condensed form of knowledge from column 2 (third column).

5 Some Limitations

In the present implementation, an agent is a Python object, and the agent’s knowledge base is still maintained as a Python list of sentences presented to a newly initialized LSTM on each call to the agent. Each run of the neural network involves presenting the sentences and the question (where “help” is one possible question) to the network and subsequently obtaining its output. Maintenance of LSTM state across calls could be used in future editions of the agent to eliminate the need for maintaining an object with a list of sentences. Hence, the “memory” of an agent is still a Python list, although a sequence-to-sequence network performs operations like distilling the list to the essentials or answering questions about the sentences in the list. Given that our focus is on agents having a (limited) understanding of their own knowledge state, which is a basic form of self-awareness, much less emphasis has been placed on attempting to achieve general propositional reasoning. An error rate of about 1% has been achieved on validation data, but the agent may perform poorly if presented with very long chains of reasoning of more than three or four sentences or with combinations of sentences that differ too much from what was seen in training. Users are invited to view the training and validation sets in the sample data at https://github.com/CoderRyan800/redesigned-octo-goggles/blob/master/logic_data_extended_00.tsv to view the kinds of problems we trained the agent to solve.

6 Demo and Source Code

There is a demonstration of this system available for peer review. In the demo, logical operators are represented as follows:

1. Logical NOT: $\sim$
2. Logical AND: $\&$
3. Logical OR: $|$
4. Logical XOR: $\wedge$
5. Implication: $\implies$
6. Reverse implication: $\impliedby$
7. If and only if: $\iff$

Each of the two agents is provided with a list of sentences, and each sentence may only contain one or two propositions. Run each cell to install the code and the data files in Part 1 of the notebook. In Part 2, look for the code cell with this code: `run_dual_agent_demo(agent_1_initial_knowledge_list = ['~A', 'B ==>A'],agent_2_initial_knowledge_list = ['B ^C'],question_for_agent_1 = 'What is C ?')` The source code is available at <https://github.com/CoderRyan800/redesigned-octo-goggles.git>. To run the demo:

1. `docker pull ryan20083437/my_project:latest`
2. `docker run -it --name my_project_container ryan20083437/my_project:latest`

We would like to give credit to two important sources for a great deal of our source code:

1. The Python repository for the well-known textbook, Artificial Intelligence: A Modern Approach [10] is located at <https://github.com/aimacode/aima-python> [9] and we have borrowed propositional logic code from that repository in order to create training and validation data for our agents. We also use code from this repo to perform syntax checks on user inputs to the agents in our web demo.
2. The Keras [4] documentation at https://keras.io/examples/lstm_seq2seq/ and at https://keras.io/examples/lstm_seq2seq_restore/ provides example code that demonstrates sequence-to-sequence models, and we have also utilized this in a somewhat modified form to create and to train our own networks.

7 Conclusion

This work demonstrates an early form of self-awareness in an LSTM-based agent, enabling it to:

- Monitor its own knowledge state and explicitly request help when uncertain.
- Detect contradictions in its reasoning and issue warnings.
- Cooperate with other agents to complete logical reasoning tasks.

While research in self-awareness and neural-symbolic reasoning was active by 2020, prior work focused on self-awareness at the neural or architectural level rather than at the knowledge-state level. Our study provides empirical validation that:

- LSTM-based agents can exhibit explicit self-awareness in symbolic reasoning.
- Multi-agent cooperation can be self-directed through knowledge-based introspection.
- Contradiction detection can be embedded into a neuro-symbolic reasoning framework.

This study represents an incremental step rather than a radical breakthrough. However, it helps bridge the gap between neural-symbolic AI and self-awareness research. Future work should investigate how these principles can be scaled to more complex reasoning tasks and explore integration with Transformer-based architectures.

8 Acknowledgments

The author would like to acknowledge OpenAI’s ChatGPT-4o for assistance in refining the structure and contextual framing of this paper, as well as Perplexity AI for providing an independent assessment of this work’s novelty relative to prior AI research. These analyses confirmed that while this study represents an incremental step in self-aware AI, its contributions in integrating knowledge-state self-awareness, agent cooperation, and contradiction detection in an LSTM-based model were unique at the time of its completion in 2020. The author also gratefully acknowledges the valuable feedback and advice received from Zaid Towfic, who pointed out several deficiencies in the first publication of this article.

References

- [1] Martín Abadi, Ashish Agarwal, Paul Barham, Eugene Brevdo, Zhifeng Chen, Craig Citro, Greg S. Corrado, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Ian Goodfellow, Andrew Harp, Geoffrey Irving, Michael Isard, Yangqing Jia, Rafal Jozefowicz, Lukasz Kaiser, Manjunath Kudlur, Josh Levenberg, Dandelion Mané, Rajat Monga, Sherry Moore, Derek Murray, Chris Olah, Mike Schuster, Jonathon Shlens, Benoit Steiner, Ilya Sutskever, Kunal Talwar, Paul Tucker, Vincent Vanhoucke, Vijay Vasudevan, Fernanda Viégas, Oriol Vinyals, Pete Warden, Martin Wattenberg, Martin Wicke, Yuan Yu, and Xiaoqiang Zheng. TensorFlow: Large-scale machine learning on heterogeneous systems, 2015. Software available from tensorflow.org.
- [2] Tarek R. Besold, Artur S. d’Avila Garcez, Sebastian Bader, Howard Bowman, Pedro M. Domingos, Pascal Hitzler, Kai-Uwe Kühnberger, Luís C. Lamb, Daniel Lowd, Priscila Machado Vieira Lima, Leo de Penning, Gadi Pinkas, Hoifung Poon, and Gerson Zaverucha. Neural-symbolic learning and reasoning: A survey and interpretation. *CoRR*, abs/1711.03902, 2017.
- [3] Tarek R. Besold and Kai-Uwe Kühnberger. Towards integrated neural-symbolic systems for human-level ai: Two research programs helping to bridge the gaps. *Biologically Inspired Cognitive Architectures*, 14:97–110, 2015.
- [4] François Chollet et al. Keras. <https://keras.io>, 2015.
- [5] B. Z. Du, Q. Guo, Y. Zhao, T. Zhi, Y. Chen, and Z. Xu. Self-aware neural network systems: A survey and new perspective. *Proceedings of the IEEE*, 108(7):1047–1067, 2020.
- [6] Richard Evans, David Saxton, David Amos, Pushmeet Kohli, and Edward Grefenstette. Can neural networks understand logical entailment? *CoRR*, abs/1802.08535, 2018.

- [7] Barbara Hammer and Pascal Hitzler, editors. *Perspectives of Neural-Symbolic Integration*. Springer-Verlag, Berlin Heidelberg, 2007.
- [8] Qiuyuan Huang, Paul Smolensky, Xiaodong He, Li Deng, and Dapeng Wu. A neural-symbolic approach to design of captcha. *arXiv*, 2017. arXiv:1710.11475v2 [cs.CL].
- [9] Stuart Russell and Peter Norvig. aima-python. <https://github.com/aimacode/aima-python>, 2019.
- [10] Stuart J. Russell and Peter Norvig. *Artificial Intelligence - A Modern Approach (3. internat. ed.)*. Pearson Education, 2010.